Computer-Aided Design of Digital Systems (CAD)

ECE536(CAD), Lecture 3:

Verilog Hardware Description Language

Mostafa E. Salehi
School of Electrical & Computer Engineering
University of Tehran
mersali@ut.ac.ir

Some lecture material are derived from MIT and other related courses



Outline

- Introduction
- History of Verilog HDL
- Overview of Digital Design with Verilog
- Verilog Syntax
- Synthesizable Verilog

Introduction

- HDL is a language that **describes the hardware of digital systems** in a textual form.
- It resembles a programming language, but specifically **describes hardware structures and behaviors**.
- The main difference with the traditional programming languages is HDL's representation of extensive **parallel operations** whereas traditional ones represents mostly serial operations.
- The most common use of a HDL is to provide an **alternative to schematics**.

Evolution of Computer-Aided Digital Design

- SSI: Small scale integration
  - A few gates on a chip
- MSI: Medium scale integration
  - Hundreds of gates on a chip
- LSI: Large scale integration
  - Thousands of gates on a chip
  - CAD: Computer-Aided Design
  - Logic and circuit simulators
  - Prototyping on bread board
  - Layout by hand (on paper or a computer terminal)

Evolution of Computer-Aided Digital Design (cont'd)

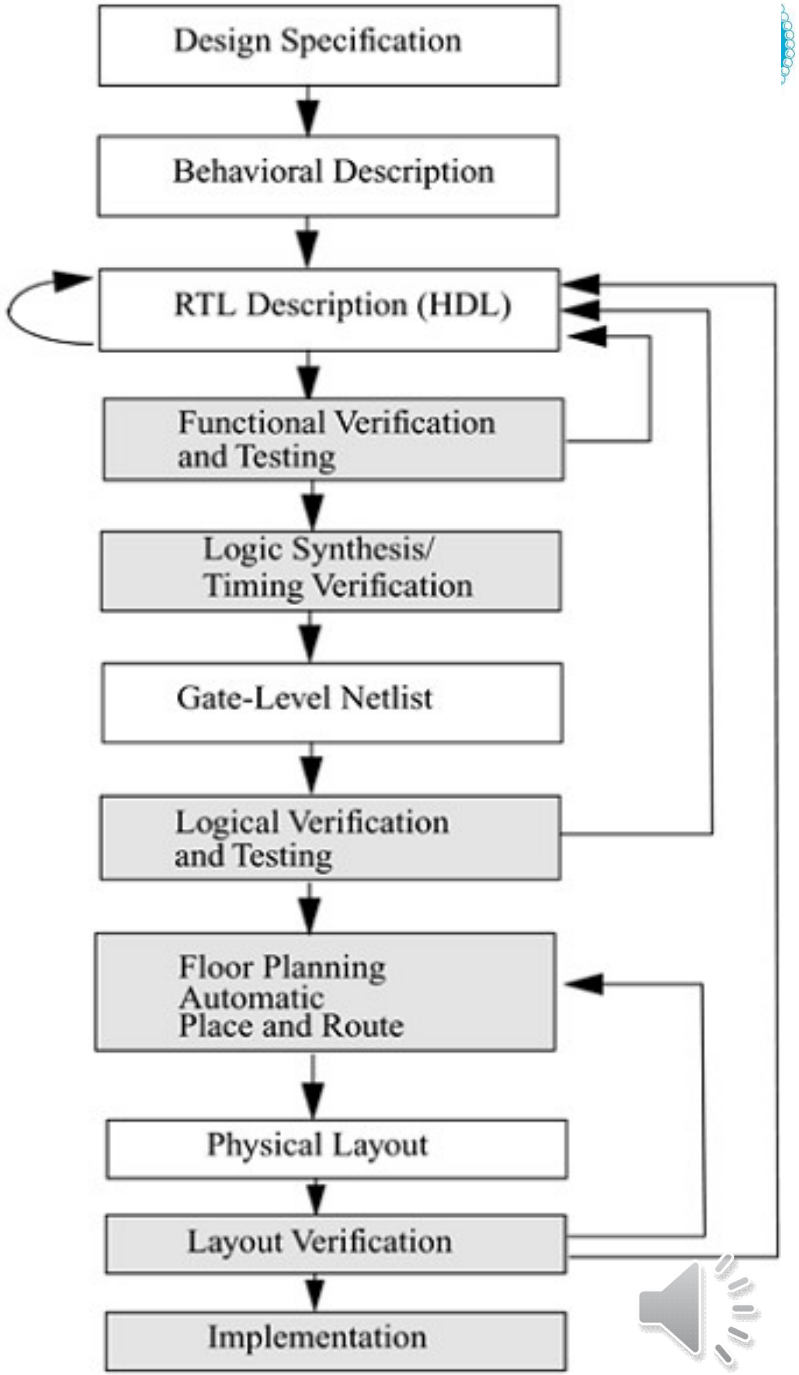
- VLSI: Very Large Scale Integration
  - Hundred thousands of gates
  - Not feasible anymore:
    - Bread boarding
    - Manual layout design
  - Simulator programs
  - Automatic place-and-route
  - Bottom-Up design
    - Design small building blocks
    - Combine them to develop bigger ones
  - More and more emphasis on logic simulation

Emergence of HDLs

- The need to a standardized language for hardware description
  - Verilog® and VHDL
- Simulators emerged
  - Usage: functional verification
  - Path to implementation: manual translation into gates
- Logic synthesis technology
  - Late 1980s
  - Dramatic change in digital design
    - Design at Register-Transfer Level (RTL) using an HDL

Typical Design Flow (in 1996)

1. Design specification
2. Behavioral description
3. RTL description
4. Functional verification and testing
5. Logic synthesis
6. Gate-level netlist
7. Logical verification and testing
8. Floor planning, automatic place & route
9. Physical layout
10. Layout verification
11. Implementation



Typical Design Flow

- Most design activity
  - In 1996:
    - Manually optimizing the RTL design
    - CAD tools take care of generating lower-level details
    - Reducing design time to months from years
  - Today
    - Still RTL is used in many cases
    - But, synthesis from behavioral-level also possible
    - Digital design now resembles high-level computer programming

Types of HDL

- There are two standard HDL's that are supported by IEEE.
  - **VHDL** (*Very-High-Speed Integrated Circuits Hardware Description Language*) - Sometimes referred to as VHSIC HDL, this was developed from an initiative by US. Dept. of Defense.
  - **Verilog HDL** – developed by Cadence Data systems and later transferred to a consortium called *Open Verilog International* (OVI).



History of Verilog® HDL

- Three factors to success of Verilog
  - Programming Language Interface (PLI)
    - Extend and customize simulation environment
  - Close attention to the needs of ASIC foundries
    - "Gateway Design Automation" partnership with Motorola, National, and UTMC in 1987-89
  - Verilog-based synthesis technology
    - "Gateway Design Automation" licensed Verilog to Synopsys
    - Synopsys introduced synthesis from Verilog in 1987

History of Verilog® HDL (cont'd)

- Today
  - Market divided between Verilog & VHDL
    - VHDL mostly in Europe
    - Verilog dominant in US
  - VHDL
    - More general language
    - Not all constructs are synthesizable
  - Verilog:
    - Not as general as VHDL
    - Most constructs are synthesizable

History of Verilog® HDL

- Beginning: 1983
  - Simulation environment
    - Comprising various levels of abstraction
      - Switch (transistors), gate, register-transfer, and higher levels
- IEEE 1364-1995
- IEEE 1364-2001
- IEEE 1364-2005

Popularity of Verilog HDL

- Verilog HDL
  - Easy to learn, easy to use
  - Similar in syntax to C
  - Allows different levels of abstraction and mixing them
  - Supported by most popular logic synthesis tools
  - Post-logic-synthesis simulation libraries by all fabrication vendors
  - PLI to customize Verilog simulators to designers' needs

Levels of Abstraction (1)

- **Switch Level:** Module implemented with switches and interconnects. Lowest level of Abstraction

- **Gate Level:** Module implemented in terms of logic gates like (and ,or) and interconnection between gates

Levels of Abstraction (2)

- **Dataflow Level**: Module designed by specifying dataflow. The designer is aware of how data flows between hardware registers and how the data is processed in the design
- **Behavioral Level :**Module can be implemented in terms of the desired design algorithm without concern for the hardware implementation details. Very similar to C programming

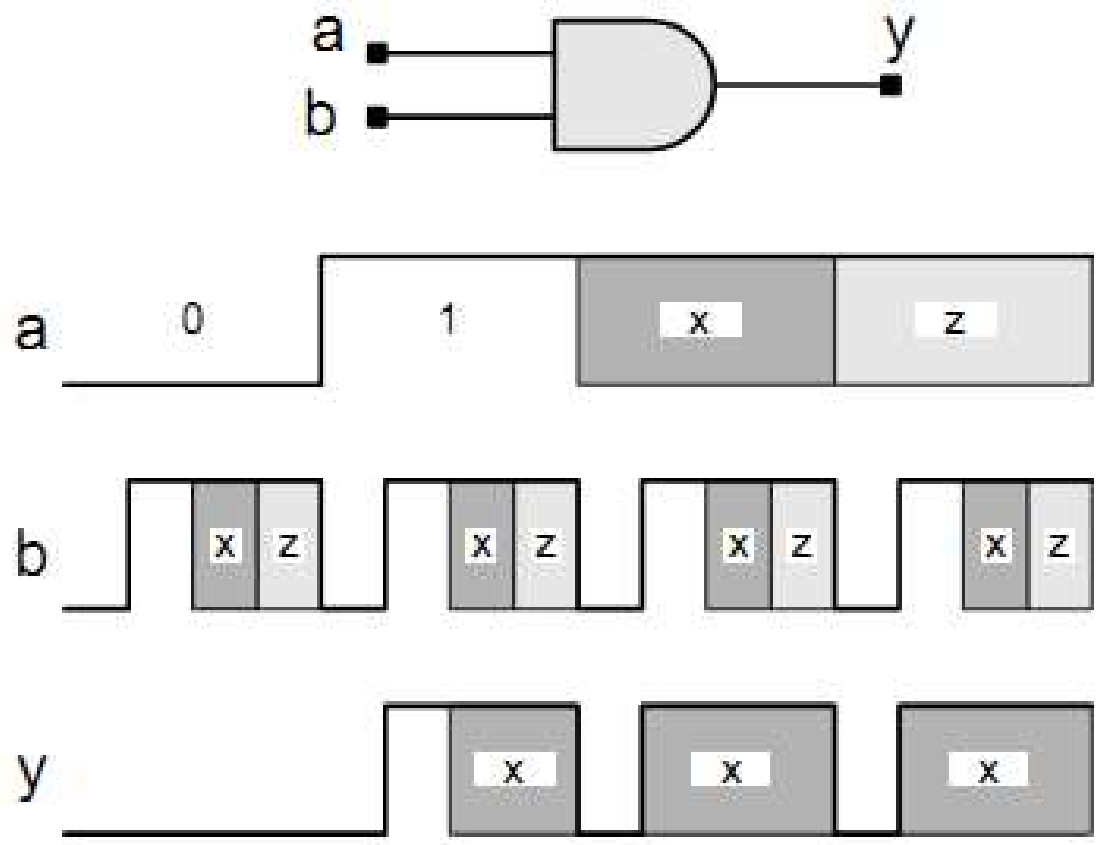
Data Types in Verilog

- Registers
- Nets
- Memories
- Integers and times
- Reals

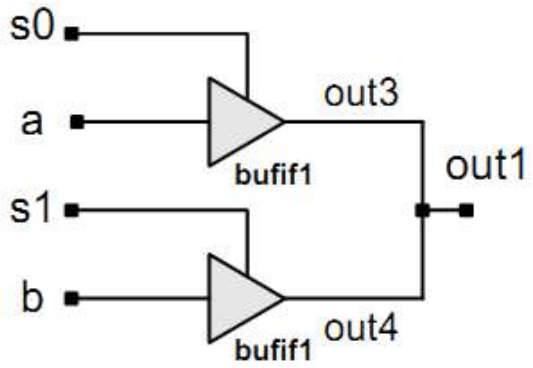
Value sets

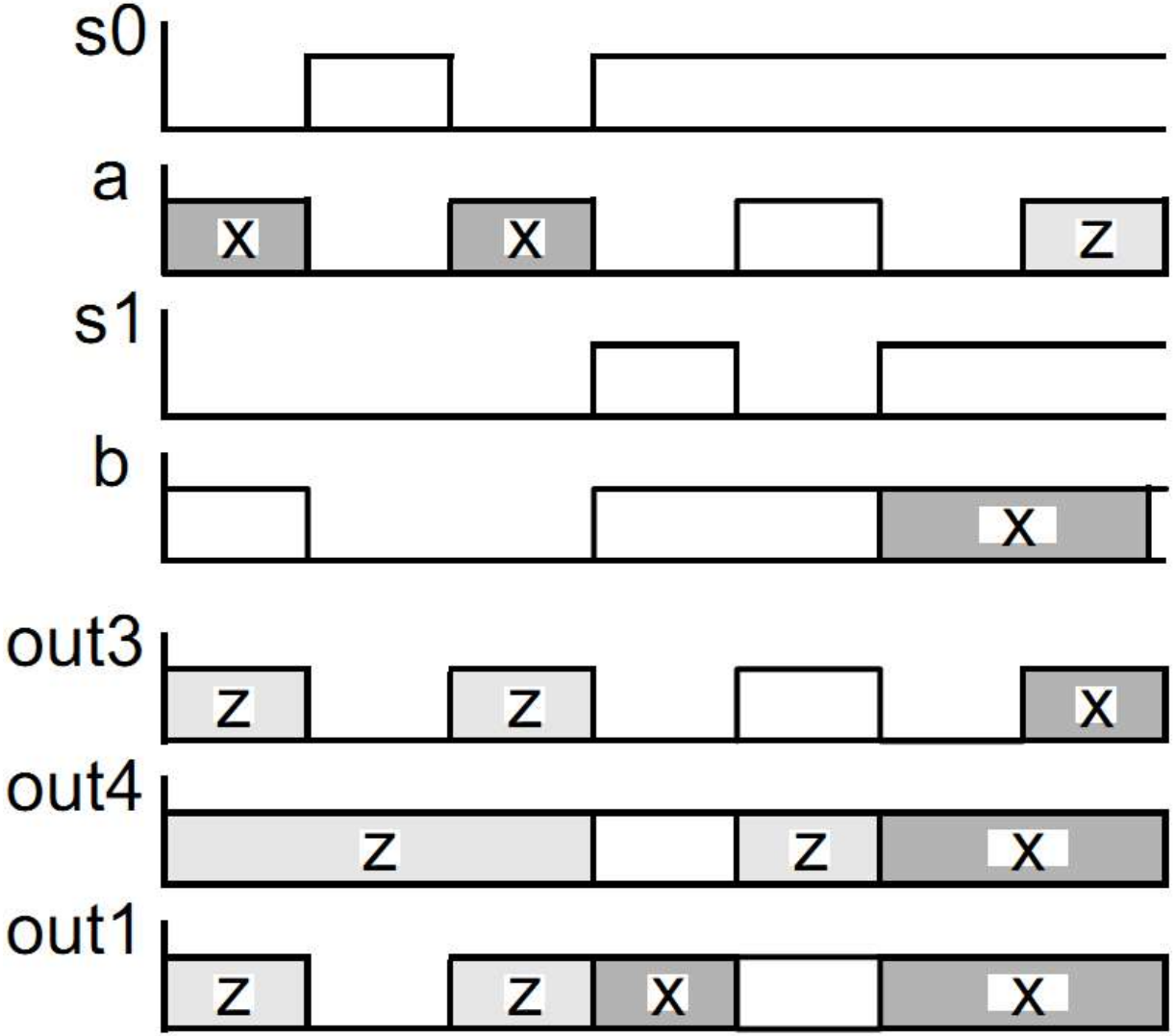
- 0: logic zero or false condition
- 1: logic one or true condition
- x: unknown logic value
- z: high-impedance state

4-value logic

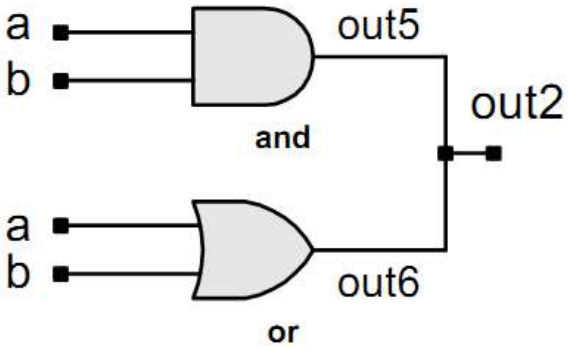


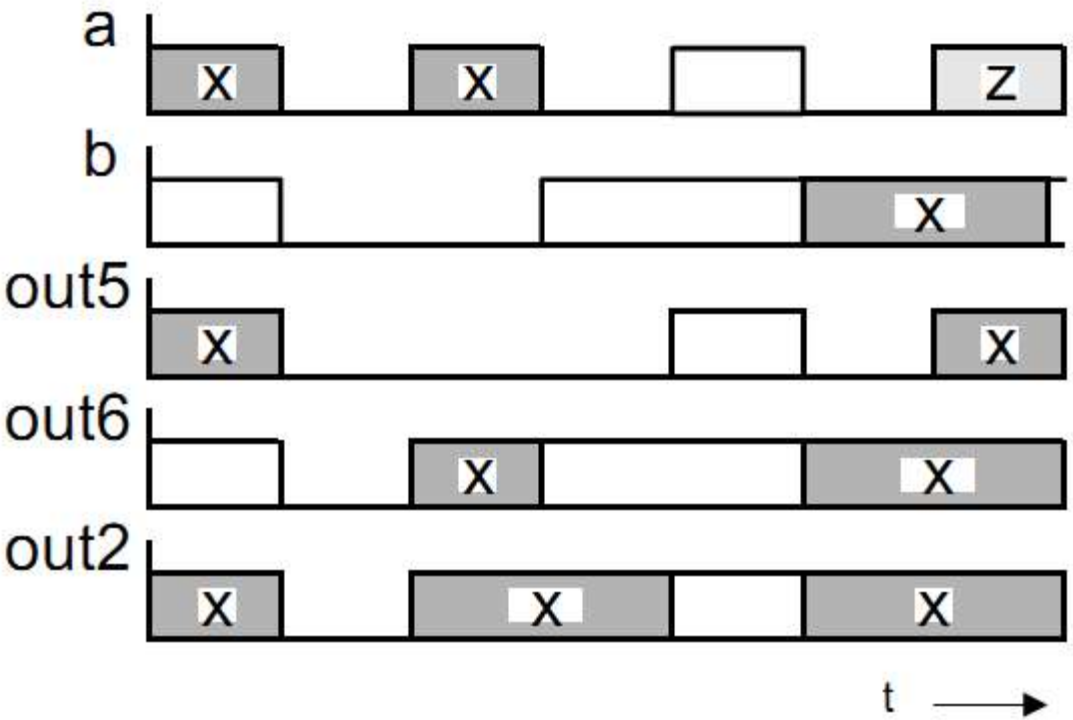
Resolution of Contention Between Drivers





Resolution of Contention Between Drivers





Number Representation

- Constant numbers can be: decimal, hexadecimal, octal, or binary
- ***<size><base_format><number>***
  - ***<size>****:*
    - *number of bits that the constant contains*
  - ***<base_format>:***
    - *a letter specifying the number's base (d, h, o, or b),* followed by the single quote character (')
  - ***<number>:***
    - *digits that are legal for the specified <base_format>*




Examples

- 4'b1001 // a 4-bit binary number
- 8'b1010_1110 // This is a 8 bit binary number
- 16'ha04f // a 16-bit hexadecimal number
- 16'D255 // This is a 16 bit decimal number
- -4'd13 // This is a 4 bit negative decimal number,
  // stores as 2's complement of 13
- 4'b01x0 // a 4-bit binary number
- 12'hx // a 12-bit unknown number
- 1234 // This is a 32 bit decimal number
- h'62 // This is a 32 bit hex number
- o'255 // This is a 32 bit octal number
- 4'b10?? // This is a 4 bit binary number equal to 4'b10xx






Verilog Registers

- In digital design, registers represent memory elements
- Digital registers need a clock to operate and update their state on certain phase or edge
- Registers in Verilog should not be confused with hardware registers
- **In Verilog, the term register (reg) simply means a variable that can hold a value**
- Verilog registers don't need a clock and don't need to be driven like a net. Values of registers can be changed anytime in a simulation by assuming a new value to the register



Registers

- Store a value from one assignment to the other
- Register type declaration examples:
  - reg a; // a scalar register
  - reg [3:0] b; // a 4-bit vector register
  - reg [2:5] c;




Nets

- Physical connections
- They do not store a value
- They must be driven by a driver (i.e., gate or continuous assignment)
- Their value is z, if not driven
- Net type declaration examples:
  - wire d; // a scalar wire
  - wire [3:0] e, f; // 4-bit vector wires
  - wire b=1'b0 // Net b is fixed to logic value 0



Arrays

<array_type> <array_name> [#first_element: #last_element]

- integer count[0:7]; // An array of 8 integer

Memories

- Memories are modeled as arrays of registers
- Each register in the array (i.e., an element or word) is addressed by a single array index
- Memories are declared in register declaration statements:
  - // a 256-word, 32-bit memory
  - reg [31:0] Imem[0:255];

Integer, Real, Time

- General purpose variable used as a counter
  - integer counter;
- Declare a real variable called delta
  - real delta;
- Define a time variable sim_time
  - time sim_time;




Parameters

- Parameters are not variables
  - they are constants
- Parameter declaration examples
  - parameter size = 8; // defines size as a constant with value 8







Expression Operations

- Binary arithmetic
- Relational
- Equality
- Logical
- Bit-wise
- Reduction
- Shift
- Concatenation
- Conditional



Arithmetic Operators

- Integer division truncates any fractional part
- The result of a modulus operation takes the sign of the first operand
- If any operand bit value is the unknown value x, then the entire result value is x
- Register data types are used as unsigned values
- Negative numbers are stored in two's complement form

Operator	Application
*	`c = a * b ; // multiply a with b`
/	`c = a / b ; // int divide a by b`
+	`sum = a + b ; // add a and b`
-	`diff = a - b ; // subtract b // from a`
%	`amodb = a % b ; // a mod(b)`

Relational Operators

- The result is a scalar value:
  - 0 if the relation is false
  - 1 if the relation is true
  - x if any of the operands has unknown x bits

Operator	Application
<	a < b // is a less than b? // return 1-bit true/false
>	a > b // is a greater than b?
>=	a >= b // is a greater than or // equal to b
<=	a <= b // is a less than or // equal to b

Equality & Identity Operators

- Operands are compared bit by bit, with zero filling if the two operands do not have the same length
- For the == and != operators
  - the result is x, if either operand contains an x or a z
- For the === and !== operators
  - bits with x and z are included in the comparison and must match for the result to be true
  - the result is always 0 or 1

Operator	Application
=	`c = a ; // assign a to c`
==	`c == a ; /* is c equal to a returns 1-bit true/false applies for 1 or 0, logic equality, using X or Z oper-ands returns always false 'hx == 'h5 returns 0 */`
!=	`c != a ; // is c not equal to // a, retruns 1-bit true/ // false logic equality`
===	`a === b ; // is a identical to // b (includes 0, 1, x, z) / // 'hx === 'h5 returns 0`
!==	`a !== b ; // is a not // identical to b returns 1- // bit true/false`

Logical Operators

- Expressions connected by && and || are evaluated from left to right
- Evaluation stops as soon as the result is known
- The result is a scalar value:
  - 0 if the relation is false
  - 1 if the relation is true
  - x if any of the operands has unknown x bits

Operator	Application

&&	`a && b ; // is a and b true?`
	`// returns 1-bit true/false`

||	`a || b ; // is a or b true?`
	`// returns 1-bit true/false`

!	`if (!a) ; // if a is not true`
	`c = b ; // assign b to c`

Unary, Bitwise and Reduction Operators

- When operands are of unequal bit length, the shorter operand is zero-filled in the most significant bit positions

Operator	Application
+	Unary plus & arithmetic(binary) addition
–	Unary negation & arithmetic (binary) sub-traction
&	`b = &a ; // AND all bits of a`
|	`b = |a ; // OR all bits`
^	`b = ^a ; // Exclusive or all bits of a`
~&, ~|, ~^	NAND, NOR, EX-NOR all bits to-gether `c = ~& b ; d = ~| a; e = ^c ;`

~, &, |, ^	bit-wise NOT, AND, OR, EX-OR
	`b = ~a ; // invert a`
	`c = b & a ; // bitwise AND a,b`
	`e = b | a ; // bitwise OR`
	`f = b ^ a ; // bitwise EX-OR`

~&, ~|, ~^	bit-wise NAND, NOR, EX-NOR
	`c = a ~& b ; d = a ~| b ;`
	`e = a ~^ b ;`

Shift and other Operators

- Concatenations are expressed using the brace characters with commas separating the expressions
  - {a, b[3:0], c, 4'b1001}
  - if a and c are 8-bit numbers, the results has 24 bits
- Unsized constant numbers are not allowed in concatenations
- Repetition multipliers that must be constants can be used:
  - {3{a}} is equivalent to {a, a, a}
- Nested concatenations are possible:
  - {b, {3{c, d}}} is equivalent to {b, c, d, c, d, c, d}

Operator	Application
<<	a << 1 ; // shift left a by 1-bit
>>	a >> 1 ; // shift right a by 1
?:	c = sel ? a : b ; /* if sel is true c = a, else c = b , ?: ternary operator */
{}	{co, sum } = a + b + ci ; /* add a, b, ci assign the overflow to co and the re-sult to sum: operator is called concatenation */
{{}}	b = {3{a}} /* replicate a 3 times, equivalent to {a, a, a} */

Operator Precedence

Operator	Precedence
+, -, !, ~ (unary)	Highest
*, / %	
+, - (binary)	
<<. >>	
<, < =, >, >=	
=, ==. !=	
===, !==	
&, ~&	
^, ^~	
|, ~|	
&&	
||	
?:	Lowest

این اسلاید ادامه دارد ....